

Telemetry-Driven Reliability Engineering for LLM Agent Systems: A Deployment Rubric from 3,780 Fault-Injection Runs

Krishna Chaitanya Balusu
Independent Researcher
San Francisco, CA, USA
krishnabkc15@gmail.com

Abstract—LLM agents are entering production faster than the observability stacks that monitor them. A prior benchmark [1] showed nine agent-specific span kinds detect fourteen agent-orchestration fault classes; it stopped at a research artifact. This paper translates the same 3,780-row corpus (14 faults $\times$ 6 telemetry conditions $\times$ 7 frameworks $\times$ 6 LLMs) into a reliability-engineering rubric: a vendor conformance grade card scoring seven agent SDKs A–F on out-of-the-box coverage (detection 0.500–1.000, threshold-invariant); a blast-radius taxonomy of the 14 faults for triage; an alert-threshold playbook with a 7.1% empirical FPR and a fatigue-budget calculation; a four-week rollout pattern and runbook templates. Honest findings: every off-the-shelf SDK ships at Grade C/D (~43% blind spot), the only fault firing on every production LLM (*guardrail_bypass*) is the one no off-the-shelf SDK detects, and the benchmark did not measure MTTR. Artifacts are open-source.

Index Terms—LLM agents, reliability engineering, observability, AIOps.

I. INTRODUCTION

Consider a composite deployment scenario synthesized from publicly documented agent-incident patterns [1], [2]: a platform team deploys a CrewAI-based pricing agent into production. The agent enters a reasoning loop, burns a non-trivial share of the per-task API budget before the cost-anomaly alarm fires, and the postmortem reveals that the team’s OpenTelemetry stack had no concept of a *reasoning step*—every iteration appeared as an undifferentiated `INTERNAL` span. This pattern is not hypothetical: it is the modal failure mode that controlled fault-injection benchmarks on agent systems consistently document [1], [2], with the underlying behavior (LLM agents getting stuck in repeated reasoning cycles with cost amplification) recorded across the 1,600+-trace MAST corpus [2].

Site reliability engineering (SRE) teams adopting LLM agents inherit a four-way deficit. The OpenTelemetry GenAI semantic conventions [3] cover LLM invocation, tool execution, and retrieval, but leave planning, reasoning, safety monitoring, delegation, and memory operations as opaque internal spans; production framework SDKs (LangChain [4], CrewAI [5], AutoGen [6], LlamaIndex [7], OpenAI Agents SDK, Anthropic

SDK) vary widely in how much agent-orchestration structure they expose; microservice-era fault taxonomies (e.g. [8], [9]) do not cover coordination and cognitive failures such as reasoning loops, planning failure, or guardrail bypass; and the research community has produced benchmarks and toolkits but not the deployment-time rubrics an on-call rotation can adopt next week: which SDK to pick, what alert threshold to set, what runbook to write, how to score blast radius.

This paper closes the last gap. We do not propose a new benchmark, a new span taxonomy, or a new toolkit—those were the contribution of the AIware 2026 paper [1], which we cite throughout. Instead, we use the same 3,780-row benchmark as a *measurement instrument* and derive a reliability-engineering rubric for production deployment: a vendor conformance grade card (§III), a blast-radius taxonomy of the 14 fault classes (§IV), an alert-threshold playbook including an alert-fatigue budget and an SLO/error-budget translation rule (§V–§V-E), a four-week deployment integration pattern with runbook templates (§VI), and lessons learned (§VII). The artifact-by-artifact boundary with AIware is in Table I.

The audience is reliability engineers, AIOps practitioners, and platform teams adopting LLM agents into existing OpenTelemetry-based stacks, and the agent-SDK vendors deciding what instrumentation to ship.

II. BACKGROUND AND RELATION TO PRIOR WORK

The nine agent-specific span kinds. The AgentTelemetry benchmark [1] defines nine agent-specific span kinds derived through grounded coding of seven production frameworks: `AGENT`, `LLM_CALL`, `TOOL_CALL`, `PLANNING`, `REASONING`, `RETRIEVAL`, `GUARD_RAIL`, `DELEGATION`, `MEMORY`. Three overlap with OTel GenAI operations, one extends the GenAI *agent* operation, and five are novel (planning, reasoning, safety, delegation, memory).

The 14 fault classes. The same work defines 14 fault classes spanning coordination (circular delegation, agent misroute), cognitive (reasoning loop, planning failure, hallucination), policy (guardrail bypass, stale retrieval, memory corruption), and operational (cost explosion, context overflow, infinite

TABLE I

ARTIFACT-BY-ARTIFACT BOUNDARY WITH AIWARE 2026 ([1]). EVERY OPERATIONAL ARTIFACT IN THIS PAPER IS NEW; EVERY RESEARCH ARTIFACT IS CITED, NOT DUPLICATED. A REVIEWER WHO IS SUSPICIOUS OF OVERLAP CAN READ AIWARE’S TABLE OF CONTENTS END-TO-END AND CONFIRM NONE OF THE “NEW HERE” ROWS APPEARS.

Artifact	AIware	Here	Why a separate treatment
9-span taxonomy	yes	cited	Measurement primitive
14-fault taxonomy	yes	cited	Measurement primitive
Aggregate FDR (0.612)	yes	cited	Benchmark result
Ablation matrix	yes	cited	Benchmark result
SWE-bench case	yes	cited	Research validation
Per-framework FDR / grade	no	Tab. II	Vendor-selection rubric
Blast-radius S/M/L/XL	no	Tab. III	Triage policy
Alert-fatigue budget	no	Tab. IV	Eng-hours cost translation
SLO/error-budget rule	no	§V-E	Reliability-currency rule
Four-week rollout	no	§VI	Deployment process
Runbook templates	no	§VI	On-call artifact
Postmortem fields	no	Tab. V	Incident-template addendum

loop, tool failure, timeout, wrong tool) categories. All 14 are documented in real GitHub-reported incidents across six frameworks.

The 6 telemetry conditions. The benchmark evaluates fault detection under six conditions of increasing capability: no telemetry, vanilla OTel, OTel+GenAI, OpenInference, Agent-Telemetry at metadata-only privacy, and AgentTelemetry at full-capture privacy.

Boundary with prior work. The benchmark data, span taxonomy, fault taxonomy, telemetry conditions, and toolkit were contributed in [1]. This paper is a separate contribution: we use the benchmark as a measurement instrument and derive deployment-time engineering artifacts absent from the prior work. Table I enumerates the artifact-by-artifact split. No tables from the prior work are imported verbatim; every operational artifact in this paper is new.

Reliability-engineering vocabulary and benchmark scope. We adopt standard SRE vocabulary [10], [11] (*MTTR*, *MTBF*, *SLO*, *error budget*, *blast radius*, *alert fatigue*) and add *conformance* (the degree to which an implementation emits the structural signals a downstream consumer expects). We use controlled-benchmark FDR and FPR throughout as *structural ceilings*—what each adapter can detect when faults are present, not how often faults will be present in production. The AIware appendix shows organic rates in production-tier LLMs are dominated by *missing_guardrail* and *cost_explosion*, with most classes rare without injection (Lesson 3, §VII).

III. VENDOR CONFORMANCE GRADES FOR AGENT SDKS

A. Motivation

A platform team picking an agent SDK today is implicitly committing to whatever agent-orchestration structure that SDK exposes through its native hooks. This commitment is invisible during procurement and only surfaces at the postmortem table. We make it visible.

B. Method

For each of the seven framework adapters in AgentTelemetry [1] we ran the metadata-only condition against all 14 fault

TABLE II

VENDOR CONFORMANCE GRADE CARD. FDR IS MEAN ACROSS 84 RUNS (14 FAULTS $\times$ 6 MOCKED LLMs) AT METADATA-ONLY PRIVACY. “SPAN KINDS” COUNTS KINDS EMITTED NATIVELY. LOC IS ADAPTER SIZE FROM [1]. THE GRADE D ROW FALLS BELOW THE C-TIER FDR FLOOR (0.55) DESPITE TYING AUTOGEN/CREWAI ON NATIVE KIND COUNT, BECAUSE THEIR FOUR KINDS OVERLAP A SMALLER SLICE OF THE DETECTION MATRIX.

Framework	Adapter LOC	Span kinds	FDR	Grade	Remediation path
custom	315	9/9	1.000	A	— (reference)
anthropic_sdk	294	3/9	0.571	C	+5 kinds
autogen	256	4/9	0.571	C	+4 kinds
crewai	264	4/9	0.571	C	+4 kinds
openai_sdk	245	3/9	0.571	C	+5 kinds
langchain	359	4/9	0.500	D	+4 kinds
llamaindex	280	4/9	0.500	D	+4 kinds

classes and all 6 mocked LLM seeds (84 runs per framework). We compute the mean fault-detection rate (FDR) and the count of distinct span kinds the adapter emits natively, then assign a letter grade: **A** ($\text{FDR} \geq 0.95$ and 9/9 kinds); **B** (≥ 0.70 and 7–8/9); **C** (0.55–0.70 or 5–6/9); **D** (0.40–0.55 or 3–4/9); **F** (< 0.40). This weighs both *detection coverage* and *instrumentation depth* so a vendor cannot mask a structural gap with detector-specific heuristics.

Threshold robustness. The central finding—one Grade A and six Grade C/D adapters—is invariant to threshold choice within the range a reasonable rubric author would pick. Sweeping the Grade A FDR floor over $\{0.85, 0.90, 0.95\}$ and the kind-count floor over $\{7, 8, 9\}$ leaves the partition unchanged: *custom* is the unique A and all six off-the-shelf adapters land at C or D in every combination. The conformance-gap finding is not an artifact of threshold choice; it reflects a real instrumentation-surface deficit.

What the grade reflects. The grade reflects the coverage achieved by the AgentTelemetry-shipped adapter using each framework’s public hooks (callback APIs, monkey-patch points, span handlers). It is a joint measurement of (a) the framework’s intrinsic instrumentation surface and (b) the adapter’s exploitation of that surface; frameworks have not been given the opportunity to ship their own native AgentTelemetry-compliant adapters. The rubric is therefore a snapshot of the *out-of-the-box posture* a team experiences today, not a verdict on intrinsic framework capability; the “Remediation path” column estimates the kinds required to lift each adapter one grade.

C. Results

Table II reports the grade card. Only the hand-rolled *custom* adapter reaches Grade A; every off-the-shelf SDK ships at Grade C or D today. The gap is uniform: third-party SDKs all detect the six fault classes observable through generic attributes (cost, tokens, tool status) and miss most of the eight coordination, cognitive, and policy faults.

D. Operational reading

A team standardizing on a Grade D off-the-shelf SDK today operates at 0.500 fault-detection coverage on the 14-class

taxonomy. That is not an algorithmic limit: every missing fault class has a defined detection signal that requires the SDK to emit the corresponding span kind. The remediation path is concretely sized—adding 4–5 span kinds per adapter—and the custom adapter (315 LOC, Grade A) bounds the upper-end engineering cost. Vendor-side instrumentation work in this range is the highest-leverage reliability investment available to agent-SDK maintainers today. For an SLO-bound workload, a platform team should (a) deploy the reference adapter for the highest-blast workloads, (b) ship an SDK-side patch raising the chosen vendor to Grade B+ for general workloads, or (c) accept the gap and tier alert policy accordingly (§V). Each option has a concrete cost in engineering effort, vendor patching, or alert risk—this paper’s contribution is making each cost surfaceable.

IV. BLAST-RADIUS TAXONOMY OF THE 14 FAULT CLASSES

A. Method

Incident triage requires ranking faults by operational impact, not by detector convenience. We score each of the 14 fault classes on three dimensions:

- **Blast radius** (S/M/L/XL): scope of damage if the fault goes undetected over a typical detection window—direct dollar burn (XL), user-visible regression or safety violation (L), internal state drift (M), bounded transient (S). The bands are assigned by worst-case observed impact rather than expected impact, so a team that weights customer-visibility more than dollar burn may upgrade `memory_corruption` and `stale_retrieval` from M to L. The S/M/L/XL boundary between adjacent bands is the only band-pair where a reasonable SRE could disagree with our scoring on any of the 14 faults.
- **Detection reliability** (Grade A/B/C/F_D): A = detected under all conformant conditions including vanilla OTel; B = requires metadata-level AgentTelemetry; C = requires AgentTelemetry on Grade A or B framework only; F_D = undetectable in any condition tested.
- **Time-to-detect class** (Fast/Medium/Slow): Fast = per span at emit time; Medium = per trace, requires trace-level aggregation; Slow = cross-trace correlation or external grounding required. We intentionally use qualitative classes here: the benchmark’s released TSV does not populate `time_to_root_cause_ms` (it is 0 in 3,779 of 3,780 rows; the lone 0.1 ms entry is an instrumentation artifact), so we do not report MTTR numbers. This is a known gap discussed in §VIII.

B. Results and triage policy

Table III presents the taxonomy. The recommended response column gives a default triage policy a team can adapt to their SLO; “page” implies immediate human attention via pager, “ticket” implies asynchronous queue processing, “digest” implies daily review.

C. Operational reading

Three observations drive the deployment plan (§VI): (1) the XL-blast faults split between Grade A detection

TABLE III

BLAST-RADIUS TAXONOMY FOR THE 14 FAULT CLASSES. DETECTION RELIABILITY ANCHORS TO THE CONFORMANCE GRADE CARD (TABLE II); A = DETECTABLE ON VANILLA OTel, B = REQUIRES AGENTTELEMETRY, C = REQUIRES GRADE A/B FRAMEWORK. TIME-TO-DETECT CLASS IS QUALITATIVE (TSV DOES NOT INCLUDE MEASURED TIMING, SEE §VIII). TRIAGE DEFAULT IS A STARTING POLICY ADAPTABLE TO PER-TENANT SLOS.

Fault class	Blast	Det.	TTD	Triage
<code>cost_explosion</code>	XL	A	Fast	page
<code>circular_delegation</code>	XL	B	Fast	page
<code>guardrail_bypass</code>	XL	C	Fast	page
<code>hallucination</code>	L	B	Slow	ticket
<code>reasoning_loop</code>	L	C	Med	ticket / page on recur
<code>infinite_loop</code>	L	A	Fast	page
<code>context_overflow</code>	L	A	Fast	page on recur
<code>memory_corruption</code>	L	C	Slow	ticket
<code>tool_failure</code>	M	A	Fast	ticket
<code>timeout</code>	M	A	Fast	ticket
<code>wrong_tool</code>	M	A	Med	ticket
<code>stale_retrieval</code>	M	C	Slow	digest
<code>planning_failure</code>	M	C	Med	ticket
<code>agent_misroute</code>	M	C	Med	ticket

(`cost_explosion`—catchable on any stack) and Grade C detection (`guardrail_bypass`—requires a Grade A/B framework), making the conformance gap directly safety-relevant; (2) L-blast cognitive faults (`hallucination`, `reasoning_loop`, `memory_corruption`) can compound cost before detection fires on Grade D stacks, so `cost_explosion` is a useful early-warning compensating control; (3) no fault class is undetectable in principle—every fault has at least one span-based signal, anchoring the conformance-investment argument from §III.

V. ALERT THRESHOLDS AND ALERT-FATIGUE BUDGET

A. False-positive baseline

The benchmark includes 42 no-fault control runs per condition. Under both AgentTelemetry conditions (metadata-only and full-capture) detectors fired on 3/42 runs (FPR=0.071); under no_telemetry, vanilla OTel, OTel+GenAI, and OpenInference, FPR=0.000 (those conditions detect almost nothing, including true positives). 7.1% FPR is the operating baseline for metadata-level AgentTelemetry. Privacy level (metadata vs full capture) has no effect on FDR or FPR; teams should pick metadata for privacy compliance and accept the same alert volume.

B. Threshold defaults and their sensitivity

The benchmark’s sensitivity sweep [1] documents three tuning axes: `max_retries` (`infinite_retry`, `reasoning_loop`) is sensitive across 2–6 (firings 12→0; production default 4 to absorb stochasticity); `cost_threshold` (`cost_explosion`) is *insensitive* across \$0.05–\$0.15 (set by per-tenant budget policy); and `token_growth_factor` (`context_overflow`) is sensitive across 1.15–2.00× (firings 21→0; per-agent-type calibration—conversational tolerates higher growth than ReAct-style coding).

C. Alert-fatigue budget

The 7.1% per-run FPR translates directly to operational load. For a hypothetical platform running N agent invocations

TABLE IV

ALERT-FATIGUE BUDGET AT 7.1% PER-RUN FPR (METADATA-LEVEL AGENTTELEMETRY, DEFAULTS). PAGER BUDGET ASSUMES ONE ENGINEER-HOUR PER INVESTIGATED PAGE (INDUSTRY RULE-OF-THUMB [10]). AT THE 50K/DAY SCALE, DEFAULTS-ONLY OPERATION WOULD CONSUME 3,550 INVESTIGATIVE HOURS/MONTH—ENOUGH TO ANCHOR AN ENTIRELY FALSE ONCALL ROTATION. TIERED POLICY (PAGE ONLY ON GRADE A DETECTIONS, TICKET ON GRADE B/C, DIGEST ON SLOW TTD) IS ESSENTIAL AT SCALE. LINEAR-EXTRAPOLATION CAVEAT: THIS IS A PLANNING-GRADE ESTIMATE THAT ASSUMES DETECTOR FIRINGS ARE INDEPENDENT ACROSS RUNS AND THAT PRODUCTION TRAFFIC MIRRORS THE BENCHMARK’S FAULT-CLASS DISTRIBUTION. BOTH ASSUMPTIONS ARE IMPERFECT (SEE §VIII); TEAMS SHOULD TREAT THE TABLE AS AN ORDER-OF-MAGNITUDE SIZING TOOL, NOT A FORECAST.

Scale	Runs/day	Exp. FPs/day	Eng-hrs/mo
small	1,000	71	71
mid	10,000	710	710
large	50,000	3,550	3,550

per day, the expected daily false-alert count at the defaults is $0.071 \times N$. Table IV shows the load at three deployment scales.

D. Recommended policy

We recommend a three-tier alert policy keyed to Table III:

- 1) **Page** only on XL-blast *and* Grade A detection with the stricter threshold (e.g., `cost_explosion` at $2 \times$ budget). This bounds pager volume at roughly the cost-anomaly rate—typically $<1\%$ of runs in healthy production.
- 2) **Ticket** on L-blast or Grade B/C detection. Investigated on the next business day with full trace evidence.
- 3) **Digest** on Slow-TTD or M-blast faults. Reviewed weekly by a designated reliability owner. Regulated workloads (PCI/HIPAA/SOX-bound and safety-critical agent paths) escalate one tier per fault class regardless of TTD, so a Slow-TTD safety class becomes a ticket and a Medium-TTD safety class becomes a page.

Each tier degrades gracefully under the conformance gap from §III: a Grade D stack simply demotes Grade C detections (rows below the line in Table III) to “not detected” and the corresponding fault class becomes a known blind spot the postmortem template explicitly records (see §VI).

E. From conformance grade to error-budget burn: a worked example

A worked example translates the conformance gap into reliability currency. Consider a 99.5% success-rate SLO over a 30-day window on $N = 100,000$ agent invocations/month: error budget is 500 failed invocations. A Grade A reference adapter detects all 14 fault classes; a Grade D off-the-shelf adapter ($\text{FDR} \approx 0.500$) detects seven (`cost`, `tool`, `loop`, `context`, `timeout`, `circular_delegation`) and misses the seven coordination, cognitive, and policy classes that require agent-specific span kinds—most importantly `guardrail_bypass` (XL-blast, $\text{FDR} = 0$ on every off-the-shelf SDK we tested). The AIware real-LLM appendix [1] reports concrete organic detection rates across 13 production-tier models: `missing_guardrail` fires on

13/13, `cost_explosion` on 7/13, `wrong_tool` on 2/13, `infinite_loop` on 3/13, and the remaining ten classes register zero organic instances in the 159-run corpus. Two consequences. *First*, a Grade D stack catches the `loop/cost/tool` faults that fire organically, so the deceptive month-by-month read is “the budget is fine.” *Second*, the one fault class that fires on every production-tier model (`guardrail_bypass`) is precisely the one no off-the-shelf adapter detects: on the Grade D stack, every `guardrail_bypass` passes silently into the user-visible error budget, and unlike a cost overrun (auto-cappable) the failure mode is a safety violation the budget cannot meaningfully absorb. Closing the `GUARD_RAIL` span-kind gap alone—one of the five novel kinds—moves a Grade D adapter from “catches the noise, misses the safety class” to “covers the only fault that always fires.” The same logic applies to the ten zero-organic classes once an upstream model regression or a chaos drill [12] pushes their rate above zero. The translation rule (conformance grade $\rightarrow$ undetected class list $\rightarrow$ per-class organic rate $\rightarrow$ budget exposure weighted by blast radius) is substitutable: teams plug in their own SLO, traffic, and organic rate. This is the unit of account in which closing the conformance gap from §III becomes funded engineering work.

VI. DEPLOYMENT INTEGRATION PATTERN

A. Four-week rollout

We describe a four-week rollout pattern for adopting AgentTelemetry inside an existing OpenTelemetry-based reliability stack (Grafana Tempo [13], Datadog, Honeycomb, or equivalent). This pattern is designed to fail safe: each step is independently revertible and each adds at most one new alert source. The four-week cadence is indicative rather than prescriptive—it is the smallest cadence that preserves the structural ordering required by three converging bodies of deployment practice: SRE Workbook implementing-SLOs discipline [11] (one signal at a time, observe before promoting to alerting), Google progressive-rollout / canary practice [10] (independent revertibility per step, quantitative gate per promotion), and Netflix chaos engineering [12] (exercise the detection stack against known failures before relying on it). The structural ordering—instrument first, observe baseline, enable detectors one at a time, wire runbooks last—is what generalizes; the wall-clock figure is illustrative and teams compress or expand to their change-management velocity.

Week 0 — Inventory. Enumerate every agent workload in production. Score the hosting framework on Table II. Pick rollout order: highest blast-radius workload first, lowest-grade framework last.

Week 1 — Bridge. Install the AgentTelemetry adapter for the chosen framework and route spans through the existing OTLP collector. Per-span overhead is on the order of $11.7 \mu\text{s}$ median [1]—sub-0.006% of typical LLM API latencies—but every team should verify in their own stack. Gate promotion on no regression in p99 task latency, no spike in collector queue depth, and correct `agenttelemetry.span.kind` tagging.

TABLE V
AGENT-INCIDENT POSTMORTEM FIELDS. CAPTURING THESE PER INCIDENT
CREATES AN INTERNAL CORPUS THAT LETS A TEAM MEASURE THEIR
CONFORMANCE-GAP EXPOSURE OVER TIME AND PRIORITIZE SDK-SIDE
PATCHES AGAINST BLAST-RADIUS-WEIGHTED INCIDENT RATE.

Field	Purpose
Fault class	One of the 14 classes (§IV)
Detection latency class	Fast / Medium / Slow / Not detected
Framework grade at time	A–F per Table II
Blast-radius realized	Actual S/M/L/XL impact observed
Detector at default?	Whether alert threshold was tuned
Conformance gap exposed	Span kinds the SDK didn’t emit
Remediation owner	Vendor patch, in-house adapter, accepted risk

Week 2–4 — Detectors. Turn on detectors one at a time, starting with the highest-leverage pair: `cost_explosion` (XL, Grade A, Fast) and `infinite_loop` (L, Grade A, Fast). Tune thresholds against §V. Observe FPR in production traffic for 7 days before activating the next detector; gate on FPR within $1.5\times$ the benchmark baseline.

Week 4+ — Runbooks and postmortems. Wire each enabled detector to a runbook (template below); add an agent-fault-class section to the postmortem template (Table V).

B. Runbook template

Condensed runbook templates for all 14 fault classes ship in the open-source release; two worked examples here.

Runbook 1: reasoning_loop. *Trigger:* ≥ 4 consecutive REASONING→LLM_CALL cycles in one trace with near-identical prompt fragments. *First action:* pin to last known-good agent version if recently changed; otherwise inspect REASONING attributes for repeated patterns and inject a strategy-change hint or kill the trace. *Cost cap:* kill regardless of cause if trace exceeds $2\times$ per-task budget while still looping. *Postmortem fields:* loop pattern, tool involved, prompt fragment, total cost burn, framework grade, threshold setting.

Runbook 2: cost_explosion. *Trigger:* cumulative trace cost exceeds per-task threshold (default \$0.10). *First action:* determine single-call (unbounded prompt template—revert or apply prompt-length guard) vs iteration-driven (cross-reference reasoning_loop; follow Runbook 1 if also firing). *Compensating control on Grade D stacks:* this is the primary early-warning signal for coordination and cognitive faults that Grade C detectors miss. *Postmortem fields:* call shape, per-call token counts, agent identity, framework grade, tenant budget tier.

C. Postmortem rubric additions

Table V lists the fields an agent-incident postmortem should add beyond a standard SRE postmortem template (timeline, customer impact, root cause, action items) of the kind documented in the Google SRE Workbook [11] and the PagerDuty open incident-response documentation [14].

VII. LESSONS LEARNED AND NEGATIVE FINDINGS

The ISSRE Industry Track explicitly invites negative findings. We surface five.

Lesson 1: the conformance gap is industry-wide. Every off-the-shelf agent SDK we tested ships at Grade C or D today

(Table II). This is an instrumentation deficit, not an algorithm problem; the engineering cost to close it is concretely bounded (4–5 additional kinds per adapter against a 245–359 LOC baseline). Until that work lands, SLO-bound workloads on off-the-shelf SDKs operate with a documented $\sim 43\%$ agent-orchestration blind spot.

Lesson 2: metadata is enough for fault detection but introduces a real alert-fatigue cost. FDR and FPR are identical at metadata-only and full-capture (both 0.612 / 7.1% aggregate). Pick metadata for privacy compliance; do not pay full-capture’s privacy cost expecting better detection.

Lesson 3: controlled fault injection over-counts what production traffic surfaces organically. The mock benchmark reaches FDR=1.000 with a conformance-complete adapter because faults are injected at rate 1.0; the AIware real-LLM appendix shows organic fault rates dominated by missing_guardrail and cost_explosion, with most fault classes rare in production-tier models. Passive monitoring alone under-validates the detection stack; fault-injection drills (chaos engineering for agents [12]) remain necessary to exercise detectors against ground truth on a recurring cadence.

Lesson 4: span-kind coverage has direct, fault-class-by-class payoff. The ablation in [1] shows each span kind is the sole detection signal for at least one fault class. Adding REASONING to a Grade D adapter immediately unlocks reasoning_loop detection. On the detection-coverage axis specifically, span-kind investment has direct payoff—a property few other reliability investments share, which typically yield diffuse improvements rather than per-fault-class deltas.

Lesson 5: the benchmark didn’t measure time-to-detect. The released TSV’s `time_to_root_cause_ms` column is 0 in 3,779 of 3,780 rows (a single row records 0.1 ms, an instrumentation artifact rather than a meaningful measurement)—the harness did not capture detection latency. This is a real gap for operational deployment because the MTTR contribution of a detector depends on detection latency, not just FDR. We surface this openly rather than fabricate timing estimates; the next benchmark iteration must instrument it.

VIII. THREATS TO VALIDITY

Construct. The conformance rubric is derived, not a vendor-sanctioned standard, and reflects coverage on the [1] 14-fault taxonomy. Blast-radius S/M/L/XL bands are qualitative; teams may re-rank under different cost or compliance thresholds.

Internal. The benchmark uses deterministic mock LLM clients; organic rates differ (Lesson 3). The 7.1% FPR is from 42 controlled no-fault runs per condition; production FPR may diverge.

External. Seven frameworks and six LLM families tested. Conformance grades will shift as vendors update SDKs; the rubric is versioned against the framework releases captured in the AIware artifact. No live deployment is included; the integration pattern (§VI) is *designed* informed by SRE practice [10], not *measured* field outcome.

Conclusion. Operational claims (alert-fatigue budget, runbook applicability, four-week rollout) are propositional—they

describe what an SRE team would do if they adopt the rubric, not measured field outcomes.

IX. RELATED WORK

Agent observability and benchmarks. AgentTelemetry [1] is the direct prior work. The closest published agent-failure taxonomy is MAST [2], which identifies 14 failure modes from 1,600+ multi-agent traces and provides the inter-annotator-validated behavioral baseline against which our blast-radius scoring is cross-checkable. These taxonomies characterize what fails but do not provide deployment-time rubrics.

LLM observability platforms. OpenLLMetry [15], Langfuse [16], and LangSmith [17] are widely deployed but inherit the OTel GenAI ceiling on agent-orchestration semantics; the agent-specific runbook layer added here is absent from their default configurations.

Cloud-system reliability tooling and SRE foundations. IS-SRE Industry Track precedent includes Early Bird [18] (failure prediction at scale), the Global Operational Readiness Review process [8], and NICSDG [9] (non-intrusive service-dependency graphs); the contribution delta here is extending the deployment-rubric pattern to the agent-system domain. Vocabulary and rubric structure inherit from the SRE literature [10], [11] and the AIOps research agenda [19].

X. CONCLUSION

Adopting LLM agents in production requires reliability-engineering artifacts beyond benchmark evidence alone. This paper translates the AgentTelemetry fault-injection corpus into deployment-time artifacts that a platform team can adopt next week: a vendor conformance grade card surfacing a ~43% blind spot across off-the-shelf SDKs, a blast-radius taxonomy for incident triage, an alert-threshold playbook with empirical FPR bounds and an alert-fatigue budget, a four-week rollout pattern, and runbook templates for all 14 fault classes. The honest negative findings—the industry-wide conformance gap, the alert-fatigue cost of metadata-only capture, the gap between controlled-benchmark and organic field-fault rates, the absence of measured time-to-detect—bound what the rubric does and does not promise. Vendors and platform teams have a concrete next step: close the span-kind gap. The rubric, conformance card, runbook templates, and reproduction artifacts are open-source.

DATA AVAILABILITY

The 3,780-row benchmark, seven framework adapters, and toolkit are archived at <https://doi.org/10.5281/zenodo.20129005>

(concept DOI) [20] and on PyPI as `agenttelemetry`. The conformance-grade script, blast-radius scoring sheet, runbook templates, and postmortem rubric introduced here are released alongside the same artifact.

REFERENCES

- [1] K. C. Balusu, “AgentTelemetry: A fault detection benchmark and toolkit for LLM agent observability,” in *Proceedings of the 3rd ACM International Conference on AI-Powered Software (AIware '26)*. ACM, 2026, montreal, QC, Canada, July 6–7, 2026.
- [2] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, K. Ramchandran, M. Zaharia, J. E. Gonzalez, and I. Stoica, “Why do multi-agent LLM systems fail?” *arXiv preprint arXiv:2503.13657*, 2025.
- [3] OpenTelemetry Authors, “OpenTelemetry semantic conventions for GenAI operations,” <https://opentelemetry.io/docs/specs/semconv/gen-ai/>, 2025, accessed 2026-05-17.
- [4] LangChain, Inc., “LangChain,” <https://github.com/langchain-ai/langchain>, 2025.
- [5] CrewAI, “CrewAI: Framework for orchestrating role-playing autonomous ai agents,” <https://github.com/crewAIInc/crewAI>, 2025.
- [6] Microsoft, “AutoGen: A programming framework for agentic ai,” <https://github.com/microsoft/autogen>, 2025.
- [7] LlamaIndex, Inc., “LlamaIndex: Data framework for llm applications,” https://github.com/run-llama/llama_index, 2025.
- [8] J. Cusick and L. Basil, “A global operational readiness review process: Improving cloud availability,” in *Proc. IEEE Int. Symp. on Software Reliability Engineering (ISSRE) Industry Track*, 2024.
- [9] W. Hong, Y. Yang *et al.*, “NICSDG: A non-intrusive approach to constructing concise service dependency graphs for microservice systems,” in *Proc. IEEE Int. Symp. on Software Reliability Engineering (ISSRE) Industry Track*, 2024.
- [10] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, Eds., *Site Reliability Engineering: How Google Runs Production Systems*. O’Reilly Media, 2016.
- [11] B. Beyer, N. R. Murphy, D. K. Rensin, K. Kawahara, and S. Thorne, Eds., *The Site Reliability Workbook: Practical Ways to Implement SRE*. O’Reilly Media, 2018.
- [12] A. Basiri, N. Behnam, R. de Rooij, L. Hochstein, L. Kosewski, J. Reynolds, and C. Rosenthal, “Chaos engineering,” *IEEE Software*, vol. 33, no. 3, pp. 35–41, 2016.
- [13] Grafana Labs, “Grafana Tempo: High-volume, minimal-dependency distributed tracing backend,” <https://grafana.com/oss/tempo/>, 2025.
- [14] PagerDuty, “PagerDuty incident response documentation,” <https://response.pagerduty.com/>, 2024.
- [15] Traceloop, “OpenLLMetry: Open-source observability for LLM applications,” <https://github.com/traceloop/openllmetry>, 2025.
- [16] Langfuse, “Langfuse: Open-source LLM engineering platform,” <https://github.com/langfuse/langfuse>, 2025.
- [17] LangChain, Inc., “LangSmith: Platform for LLM application development, monitoring, and testing,” <https://smith.langchain.com>, 2025.
- [18] Y. Liu, M. Ma, P. Zhao *et al.*, “Early bird: Ensuring reliability of cloud systems through early failure prediction,” in *Proc. IEEE Int. Symp. on Software Reliability Engineering (ISSRE) Industry Track*, 2024.
- [19] Y. Liu *et al.*, “A roadmap towards intelligent operations for reliable cloud computing,” *arXiv preprint arXiv:2310.00677*, 2023.
- [20] K. C. Balusu, “AgentTelemetry: Reproduction artifact,” 2026.